Missing feature test macros for C++20 core papers

Document #:	P2493R0
Date:	2021-11-15
Project:	Programming Language C++
Audience:	CWG, SG10

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

As Jonathan Wakely pointed out on the SG10 mailing list, neither [P0848R3] (Conditionally Trivial Special Member Functions) nor [P1330R0] (Changing the active member of a union inside `constexpr`) provided a feature-test macro.

In order to implement [P2231R1] (Missing constexpr in `std::optional` and `std::variant`), the standard library needs to know whether these features are available to know when it can use them in the library. And currently, there is no way of doing so.

Even though some compilers have implemented these features for a while, others haven't, so it's better to provide the feature-test macro now (when it will have some false negatives) than to not provide it at all (in which case libstdc++ needs to choose between never providing the library feature or causing unsupported compilers to fail even if users weren't trying to use the new library feature).

As far as choice of value goes, P0848R3 was adopted in Cologne (201907) and P1330R0 in San Diego (201811), and there is already currently a `201907L` value in the history of `__cpp_constexpr` [SD6] (a value which includes [P1331R2] and [P1668R1]) and `__cpp_concepts` (including [P1452R2]). Richard Smith suggests that we should either bump both macros to `201908L` (as just *some* larger value) or both to `202002L` (to mean the complete set of C++20 proposals).

§ 2 Proposal

This paper proposes Richard's second suggestion: bump `__cpp_concepts` and `__cpp_constexpr` to `202002L` to include P0848R3 and P1330R0, respectively. For `__cpp_concepts`, this is a new value that

requires a wording change. For `__cpp_constexpr`, there is already a later value as of `202110L` a result of [P2242R3] (gcc already implements this change and reports this new macro version, but gcc also already implements P1330R0 so this is okay. clang and msvc do not yet report this higher value).

2.1 Wording

Change 15.11 [cpp.predefined], table 21:

```
__cpp_concepts      201907L 202002L
```

Note that no change to `__cpp_constexpr` is visible in the working draft, but it will appear in SD-FeatureTest [SD6].

3 References

[P0848R3] Barry Revzin, Casey Carter. 2019-07-28. Conditionally Trivial Special Member Functions.
https://wg21.link/p0848r3

[P1330R0] Louis Dionne, David Vandevoorde. 2018-11-10. Changing the active member of a union inside constexpr.
https://wg21.link/p1330r0

[P1331R2] CJ Johnson. 2019-07-26. Permitting trivial default initialization in constexpr contexts.
https://wg21.link/p1331r2

[P1452R2] Hubert Tong. 2019-07-18. On the non-uniform semantics of return-type-requirements.
https://wg21.link/p1452r2

[P1668R1] Erich Keane. 2019-07-17. Enabling constexpr Intrinsics By Permitting Unevaluated inline-assembly in constexpr Functions.
https://wg21.link/p1668r1

[P2231R1] Barry Revzin. 2021-02-12. Add further constexpr support for optional/variant.
https://wg21.link/p2231r1

[P2242R3] Ville Voutilainen. 2021-07-13. Non-literal variables (and labels and gotos) in constexpr functions.
https://wg21.link/p2242r3

[SD6] SG10 Feature Test Recommendations.
https://wg21.link/sd6